

What Is Timsort?

Timsort is a hybrid, stable sorting algorithm derived from merge sort and insertion sort. It was created by Tim Peters in 2002 for Python and has world data by identifying and merging runs—subsequences that are already sorted. It’s also adopted in Java for non-primitive arrays, Android’s runtime, GNU Octave, V8, Swift, and Rust environments ¹.

Key Characteristics

- Hybrid of insertion sort and merge sort
- Stable: equal elements retain their original order
- Adaptive to “natural runs” in input data, minimizing comparisons and moves ¹

How Timsort Works

1. Identify Runs
 - Scan the array to find subsequences already in ascending or descending order (natural runs).
 - Ensure each run meets a minimum length (called the “minrun”) by extending short runs with insertion sort ².
2. Sort Runs
 - Apply insertion sort to each run. This is efficient for small, mostly sorted subsequences and helps prepare them for merging ³.
3. Merge Runs
 - Push runs onto a stack and merge adjacent runs when certain size-based invariants are violated.
 - Use a modified merge procedure (including “galloping” mode) to accelerate merging when one run heavily outweighs the other ¹.

Complexity Analysis

Case	Time Complexity	Space Complexity	Stable	In-Place	
Best	O(n)	O(n)	Yes	No	
Average	O(n log n)	O(n)	Yes	No	
Worst	O(n log n)	O(n)	Yes	No	

Timsort’s adaptive nature often yields performance superior to quicksort and mergesort on many real-world inputs